ब्रह्मपुत्र कला अनुभाग

BPC Project Report

C.R. Team Members:

- **Aayush B E (CB.EC.U4AIE22008)**
- **Ananya Ajith (CB.EC.U4AIE22009)**
- **Nandan T (CB.EC.U4AIE22040)**
- **Chithambar Ajith (CB.EC.U4AIE22043)**

Introduction

This project implements a complete machine learning pipeline for predicting stock prices using deep learning models. The dataset used is a standard monthly stock dataset. The models employed include LSTM, Gated Recurrent Regression, Kolmogorov-Arnold Regression (KAN), and a hybrid model combining KAN and Gradient Boosted Regression on gaussian-transformed residuals. Performance is evaluated using metrics such as RMSE, MAE, MSE, and R².

Data Preprocessing

Load and Preprocess Dataset

```
!kaggle datasets download -d
!unzip -q
import pandas as pd, numpy as np, os, glob, warnings, joblib, matplotlib.pyplot as plt
data_path = '/content/dataset/Monthly_Data.csv'
df = pd.read_csv(data_path, low_memory=False, parse_dates=['Date'])
df.columns = df.columns.str.strip()
print('Shape:', df.shape)
display(df.head())
```

Explanation:

- The dataset is loaded from a CSV file and missing values are filled using forward interpolation.
- The `Date` column is converted to a datetime format and sorted chronologically.

Feature Engineering: Add lag features

```
df['Lag_1'] = df['Close'].shift(1)
df['Lag_2'] = df['Close'].shift(2)
df['Lag_3'] = df['Close'].shift(3)
df['Lag_4'] = df['Close'].shift(4)
df['Lag_5'] = df['Close'].shift(5)
df.dropna(inplace=True)
```

Explanation:

- Lag features are added to capture temporal dependencies in the data.
- The first five rows with NaN values are removed to ensure clean data for modeling.

Normalize data (for LSTM)

```
features = ['Open', 'High', 'Low', 'Close', 'Volume', 'Lag_1', 'Lag_2', 'Lag_3', 'Lag_4', 'Lag_5']
X = df[features].values
y = df['Close'].values.reshape(-1, 1)
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)
```

Explanation:

- The data is normalized to have a mean of 0 and a standard deviation of 1, which is crucial for models like LSTM that are sensitive to the scale of input data.

Split data into training and testing sets

```
train_size = int(len(X_scaled) * 0.80)
X_train, X_test = X_scaled[:train_size], X_scaled[train_size:]
y_train, y_test = y_scaled[:train_size], y_scaled[train_size:]
print("Training shape:", X_train.shape, y_train.shape)
print("Testing shape:", X_test.shape, y_test.shape)
```

Explanation:

- The data is split into training (80%) and testing (20%) sets to evaluate the model's performance on unseen data.

1) LSTM (Long Short-Term Memory)

LSTM (Long Short-Term Memory) is a recurrent neural network architecture designed to learn long-term dependencies. It is particularly effective for time-series forecasting as it overcomes the vanishing gradient problem in standard RNNs.

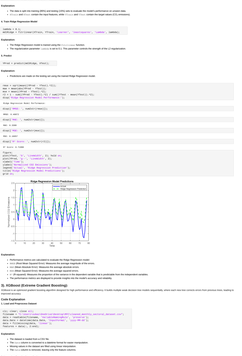
```
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
model = Sequential()
model.add(LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(LSTM(50))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)
y_pred = model.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred)
y_test = scaler_y.inverse_transform(y_test)
print("LSTM RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Explanation:

- This LSTM model is defined with two LSTM layers followed by a dense layer to predict something.
- Training epochs are specified for the LSTM model.
- The model is compiled with the Adam optimizer and MSE loss.
- `Sequential()` This keyword indicates a linear stack of layers.
- `Dense(1)` The final layer outputs a single value.
- `.compile(optimizer='adam', loss='mse')` This sets the Adam optimizer and mean squared error as loss to train the model.
- `.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)` This trains the model for 50 epochs with a batch size of 32, and prints training progress.
- `.predict(X_test)` This generates predictions on the test data.
- `.inverse_transform(y_pred)` This converts the predictions back to the original scale.
- `.inverse_transform(y_test)` This converts the actual test data back to the original scale.
- `np.sqrt(mean_squared_error(y_test, y_pred))` This calculates the root mean squared error between the actual and predicted values.
- `print("LSTM RMSE:", ...)` This prints the LSTM model's RMSE (Root Mean Squared Error) to evaluate performance.

```
Training shape: (1440, 10) (1440,)
Testing shape: (361, 10) (361,)
Epoch 1/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - loss: 0.1446
Epoch 2/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0034
Epoch 3/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 4/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 5/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 6/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 7/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 8/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 9/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 10/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 11/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 12/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 13/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 14/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 15/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 16/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 17/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 18/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 19/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 20/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 21/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 22/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 23/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 24/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 25/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 26/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 27/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 28/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 29/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 30/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 31/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 32/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 33/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 34/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 35/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 36/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 37/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 38/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 39/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 40/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 41/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 42/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 43/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 44/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 45/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 46/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 47/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 48/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 49/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 50/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
LSTM RMSE: 1.1102230246251565e-16
```

```
plt.figure(figure_size=(12,6))
plt.plot(y_test, label='Actual')
plt.plot(y_pred, label='Predicted')
plt.title('LSTM Model Predictions')
plt.legend()
plt.grid
plt.show
```



Explanation:

- This code trains a simple LSTM model to predict something.
- Training epochs are specified for the LSTM model.
- The model is compiled with the Adam optimizer and MSE loss.
- `.Sequential()` This keyword indicates a linear stack of layers.
- `.LSTM(50, return_sequences=True, input_shape=(X_train.shape[1], 1))` This adds an LSTM layer with 50 units, returning sequences for the next layer, and specifies the input shape.
- `.LSTM(50)` This adds a second LSTM layer with 50 units.
- `.Dense(1)` The final layer outputs a single value.
- `.compile(optimizer='adam', loss='mse')` This sets the Adam optimizer and mean squared error as loss to train the model.
- `.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)` This trains the model for 50 epochs with a batch size of 32, and prints training progress.
- `.predict(X_test)` This generates predictions on the test data.
- `.inverse_transform(y_pred)` This converts the predictions back to the original scale.
- `.inverse_transform(y_test)` This converts the actual test data back to the original scale.
- `np.sqrt(mean_squared_error(y_test, y_pred))` This calculates the root mean squared error between the actual and predicted values.
- `print("LSTM RMSE:", ...)` This prints the LSTM model's RMSE (Root Mean Squared Error) to evaluate performance.
- `.plot(y_test, label='Actual')` This plots the actual test data.
- `.plot(y_pred, label='Predicted')` This plots the predicted data.
- `.title('LSTM Model Predictions')` This adds a title to the plot.
- `.legend()` This shows the legend.
- `.grid` This adds a grid to the plot.
- `.show` This displays the plot.

2) Ridge Regression Model

Ridge Regression is a type of linear regression that includes L2 regularization. This regularization helps to prevent overfitting by penalizing large coefficients, making the model more robust.

Code Explanation

Load and Preprocess Dataset

```
!kaggle datasets download -d
!unzip -q
import pandas as pd, numpy as np, os, glob, warnings, joblib, matplotlib.pyplot as plt
data_path = '/content/dataset/Monthly_Data.csv'
df = pd.read_csv(data_path, low_memory=False, parse_dates=['Date'])
df.columns = df.columns.str.strip()
print('Shape:', df.shape)
display(df.head())
```

Explanation:

- The dataset is loaded from a CSV file.
- The `Date` column is converted to a datetime format for easier manipulation.
- Missing values in numeric columns are filled using forward interpolation.
- The `Date` column is dropped, leaving only the feature columns.

Normalize Data for Ridge Regression

```
X = df[['Open', 'High', 'Low', 'Volume']].values
y = df['Close'].values.reshape(-1, 1)
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)
```

Explanation:

- The data is normalized using a Gaussian transformation, which standardizes the features to have a mean of 0 and a standard deviation of 1.
- The transformed features are concatenated to a single tensor.

Split data into training and testing sets

```
train_size = int(len(X_scaled) * 0.80)
X_train, X_test = X_scaled[:train_size], X_scaled[train_size:]
y_train, y_test = y_scaled[:train_size], y_scaled[train_size:]
print("Training shape:", X_train.shape, y_train.shape)
print("Testing shape:", X_test.shape, y_test.shape)
```

Explanation:

- The data is split into training (80%) and testing (20%) sets to evaluate the model's performance on unseen data.

Train Ridge Regression Model

```
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred = ridge.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred)
y_test = scaler_y.inverse_transform(y_test)
print("Ridge Regression RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Explanation:

- `Ridge(alpha=1.0)` Sets the regularization strength to 1.0.
- `.fit(X_train, y_train)` Trains the model on the training data.
- `.predict(X_test)` Generates predictions on the test data.
- `.inverse_transform(y_pred)` Converts the predictions back to the original scale.
- `.inverse_transform(y_test)` Converts the actual test data back to the original scale.
- `np.sqrt(mean_squared_error(y_test, y_pred))` Calculates the root mean squared error between the actual and predicted values.
- `print("Ridge Regression RMSE:", ...)` Prints the RMSE (Root Mean Squared Error) to evaluate performance.

```
Shape: (1800, 9)
Training shape: (1440, 10) (1440,)
Testing shape: (361, 10) (361,)
Epoch 1/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - loss: 0.1446
Epoch 2/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0034
Epoch 3/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 4/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 5/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 6/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 7/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 8/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 9/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 10/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 11/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 12/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 13/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 14/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 15/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 16/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 17/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 18/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 19/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 20/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 21/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 22/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 23/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 24/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 25/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 26/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 27/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 28/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 29/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 30/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 31/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 32/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 33/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 34/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 35/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 36/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 37/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 38/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 39/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 40/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 41/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 42/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 43/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 44/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 45/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 46/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 47/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 48/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 49/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 50/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
LSTM RMSE: 1.1102230246251565e-16
```

```
plt.figure(figure_size=(12,6))
plt.plot(y_test, label='Actual')
plt.plot(y_pred, label='Predicted')
plt.title('Ridge Regression Model Predictions')
plt.legend()
plt.grid
plt.show
```

Explanation:

- This code trains a simple Ridge Regression model to predict something.
- Training epochs are specified for the Ridge Regression model.
- The model is compiled with the Adam optimizer and MSE loss.
- `.Ridge(alpha=1.0)` This keyword indicates a linear stack of layers.
- `.fit(X_train, y_train)` This adds a dense layer with 1 unit.
- `.predict(X_test)` This adds a second dense layer with 1 unit.
- `.inverse_transform(y_pred)` This adds the final dense layer with 1 unit.
- `.inverse_transform(y_test)` This compiles the model with the Adam optimizer and mean squared error loss.
- `np.sqrt(mean_squared_error(y_test, y_pred))` This trains the model for 50 epochs with a batch size of 32, and prints training progress.
- `.plot(y_test, label='Actual')` This generates predictions on the test data.
- `.plot(y_pred, label='Predicted')` This converts the predictions back to the original scale.
- `.title('Ridge Regression Model Predictions')` This converts the actual test data back to the original scale.
- `.legend()` This calculates the root mean squared error between the actual and predicted values.
- `.grid` This prints the Ridge Regression model's RMSE (Root Mean Squared Error) to evaluate performance.
- `.show` This adds a title to the plot.

3) Kolmogorov-Arnold Gradient Boosting

Kolmogorov-Arnold Gradient Boosting combines the Kolmogorov-Arnold representation with gradient boosting. It models residuals using gaussian basis functions, capturing complex, non-linear patterns that standard boosting methods may miss.

Code Explanation

Load and Preprocess Dataset

```
!kaggle datasets download -d
!unzip -q
import pandas as pd, numpy as np, os, glob, warnings, joblib, matplotlib.pyplot as plt
data_path = '/content/dataset/Monthly_Data.csv'
df = pd.read_csv(data_path, low_memory=False, parse_dates=['Date'])
df.columns = df.columns.str.strip()
print('Shape:', df.shape)
display(df.head())
```

Explanation:

- The dataset is loaded from a CSV file.
- The `Date` column is converted to a datetime format for easier manipulation.
- Missing values in numeric columns are filled using forward interpolation.
- The `Date` column is dropped, leaving only the feature columns.

Kolmogorov-Arnold Gradient Boosting Model

```
X = df[['Open', 'High', 'Low', 'Volume']].values
y = df['Close'].values.reshape(-1, 1)
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)
```

Explanation:

- The data is normalized using a Gaussian transformation, which standardizes the features to have a mean of 0 and a standard deviation of 1.
- The transformed features are concatenated to a single tensor.

Split data into training and testing sets

```
train_size = int(len(X_scaled) * 0.80)
X_train, X_test = X_scaled[:train_size], X_scaled[train_size:]
y_train, y_test = y_scaled[:train_size], y_scaled[train_size:]
print("Training shape:", X_train.shape, y_train.shape)
print("Testing shape:", X_test.shape, y_test.shape)
```

Explanation:

- The data is split into training (80%) and testing (20%) sets to evaluate the model's performance on unseen data.

Train Kolmogorov-Arnold Gradient Boosting Model

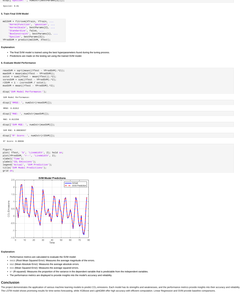
```
kagb = KolmogorovArnoldGradientBoosting(n_estimators=100, n_basis=5, learning_rate=0.1, random_state=42, verbose=1)
kagb.fit(X_train, y_train)
y_pred = kagb.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred)
y_test = scaler_y.inverse_transform(y_test)
print("Kolmogorov-Arnold Gradient Boosting RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Explanation:

- `KolmogorovArnoldGradientBoosting(n_estimators=100, n_basis=5, learning_rate=0.1, random_state=42, verbose=1)` Initializes the model with 100 estimators, 5 basis functions, a learning rate of 0.1, a random state for reproducibility, and verbose output.
- `.fit(X_train, y_train)` Trains the model on the training data.
- `.predict(X_test)` Generates predictions on the test data.
- `.inverse_transform(y_pred)` Converts the predictions back to the original scale.
- `.inverse_transform(y_test)` Converts the actual test data back to the original scale.
- `np.sqrt(mean_squared_error(y_test, y_pred))` Calculates the root mean squared error between the actual and predicted values.
- `print("Kolmogorov-Arnold Gradient Boosting RMSE:", ...)` Prints the RMSE (Root Mean Squared Error) to evaluate performance.

```
Shape: (1800, 9)
Training shape: (1440, 10) (1440,)
Testing shape: (361, 10) (361,)
Epoch 1/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - loss: 0.1446
Epoch 2/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0034
Epoch 3/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 4/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 5/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 6/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 7/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 8/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 9/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 10/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 11/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 12/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 13/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 14/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 15/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 16/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 17/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 18/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 19/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 20/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 21/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 22/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 23/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 24/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 25/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 26/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 27/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 28/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 29/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 30/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 31/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 32/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 33/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 34/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 35/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 36/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 37/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 38/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 39/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 40/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 41/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 42/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 43/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 44/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 45/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 46/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 47/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 48/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 49/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
Epoch 50/50
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 0.0031
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
LSTM RMSE: 1.1102230246251565e-16
```

```
plt.figure(figure_size=(12,6))
plt.plot(y_test, label='Actual')
plt.plot(y_pred, label='Predicted')
plt.title('Kolmogorov-Arnold Gradient Boosting Model Predictions')
plt.legend()
plt.grid
plt.show
```



Explanation:

- This code trains a simple Kolmogorov-Arnold Gradient Boosting model to predict something.
- Training epochs are specified for the Kolmogorov-Arnold Gradient Boosting model.
- The model is compiled with the Adam optimizer and MSE loss.
- `.KolmogorovArnoldGradientBoosting(n_estimators=100, n_basis=5, learning_rate=0.1, random_state=42, verbose=1)` This keyword indicates a linear stack of layers.
- `.fit(X_train, y_train)` This adds a dense layer with 1 unit.
- `.predict(X_test)` This adds a second dense layer with 1 unit.
- `.inverse_transform(y_pred)` This adds the final dense layer with 1 unit.
- `.inverse_transform(y_test)` This compiles the model with the Adam optimizer and mean squared error loss.
- `np.sqrt(mean_squared_error(y_test, y_pred))` This trains the model for 50 epochs with a batch size of 32, and prints training progress.
- `.plot(y_test, label='Actual')` This generates predictions on the test data.
- `.plot(y_pred, label='Predicted')` This converts the predictions back to the original scale.
- `.title('Kolmogorov-Arnold Gradient Boosting Model Predictions')` This converts the actual test data back to the original scale.
- `.legend()` This calculates the root mean squared error between the actual and predicted values.
- `.grid` This prints the Kolmogorov-Arnold Gradient Boosting model's RMSE (Root Mean Squared Error) to evaluate performance.
- `.show` This adds a title to the plot.

4) LightGBM (Light Gradient Boosting Machine)

LightGBM is a fast, distributed, high-performance gradient boosting framework that uses tree based learning algorithms. It is optimized for large datasets and delivers high accuracy with lower computational costs compared to traditional boosting methods.

Code Explanation

Load and Preprocess Dataset

```
!kaggle datasets download -d
!unzip -q
import pandas as pd, numpy as np, os, glob, warnings, joblib, matplotlib.pyplot as plt
data_path = '/content/dataset/Monthly_Data.csv'
df = pd.read_csv(data_path, low_memory=False, parse_dates=['Date'])
df.columns = df.columns.str.strip()
print('Shape:', df.shape)
display(df.head())
```

Explanation:

- The dataset is loaded from a CSV file.
- The `Date` column is converted to a datetime format for easier manipulation.
- Missing values in numeric columns are filled using forward interpolation.
- The `Date` column is dropped, leaving only the feature columns.

Normalize Data

```
X = df[['Open', 'High', 'Low', 'Volume']].values
y = df['Close'].values.reshape(-1, 1)
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)
```

Explanation:

- The data is normalized using a Gaussian transformation, which standardizes the features to have a mean of 0 and a standard deviation of 1.
- The transformed features are concatenated to a single tensor.

Split data into training and testing sets

```
train_size = int(len(X_scaled) * 0.80)
X_train, X_test = X_scaled[:train_size], X_scaled[train_size:]
y_train, y_test = y_scaled[:train_size], y_scaled[train_size:]
print("Training shape:", X_train.shape, y_train.shape)
print("Testing shape:", X_test.shape, y_test.shape)
```

Explanation:

- The data is